

**Auteur :**SEBASTIAO RODRIGUES Joao Paulo

**Date :** Avril 2026

**Version** 0.0.1

# Documentation et rapport du projet MDD



**MDD — Monde de Dev**

Réseau social pour développeurs

# Sommaire

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>1. Présentation générale du projet</b>          | <b>2</b>  |
| 1.1 Objectifs du projet                            | 2         |
| 1.2 Périmètre fonctionnel                          | 2         |
| <b>2. Architecture et conception technique</b>     | <b>4</b>  |
| 2.1 Schéma global de l'architecture                | 4         |
| Architecture back-end (Spring Boot)                | 5         |
| 2.2 Choix techniques                               | 6         |
| 2.3 API et schémas de données                      | 8         |
| <b>3. Tests, performance et qualité</b>            | <b>10</b> |
| 3.1 Stratégie de test                              | 10        |
| 3.2 Rapport de performance et optimisation         | 12        |
| 3.3 Revue technique                                | 13        |
| <b>4. Documentation utilisateur et supervision</b> | <b>13</b> |
| 4.1 FAQ utilisateur                                | 13        |
| 4.2 Supervision et tâches déléguées à l'IA         | 15        |
| <b>5. Annexes</b>                                  | <b>16</b> |
| Annexe 1 — Captures d'écran de l'UI                | 16        |
| Annexe 2 — Analyse des besoins front-end           | 21        |
| Annexe 3 — Définition des données                  | 23        |
| Annexe 4 — Rapport de couverture et de tests       | 24        |

# 1. Présentation générale du projet

## 1.1 Objectifs du projet

MDD (Monde de Dev) est un réseau social interne à l'entreprise développé pour la société ORION. L'application sera destinée aux développeurs et profils techniques. L'application a pour objectif de servir d'espace d'échange où les équipes techniques peuvent partager des articles autour de thèmes liés à la programmation informatique, commenter les publications de leurs collègues, et se constituer un fil d'actualité personnalisé basé sur des abonnements en fonction de différents thèmes.

Le projet répond à plusieurs besoins métiers :

- Favoriser le partage de connaissances techniques au sein des équipes ORION.
- Permettre à chaque développeur de suivre les sujets qui l'intéressent (JavaScript, Python, DevOps, etc.).
- Offrir un espace de discussion structuré autour d'articles thématiques.
- Livrer un MVP (Produit Minimum Viable) fonctionnel, stable, sécurisé, maintenable et présentable.

Le projet s'inscrit dans une démarche full-stack Angular + Spring Boot, avec une base de données relationnelle MySQL, une authentification sécurisée par JWT, et une couverture de tests d'au moins 70%.

## 1.2 Périmètre fonctionnel

Ci-dessous, nous avons une synthèse des différentes fonctionnalités développées dans le cadre du MVP :

| Fonctionnalités               | Description                                                                          | Statut |
|-------------------------------|--------------------------------------------------------------------------------------|--------|
| Inscription utilisateur       | Formulaire avec email, username, mot de passe (8+ chars, maj, min, chiffre, spécial) | Fait   |
| Connexion (email ou username) | Authentification avec persistance de session via JWT dans localStorage               | Fait   |
| Consultation du profil        | Affichage email, username et liste des abonnements                                   | Fait   |

|                         |                                                              |      |
|-------------------------|--------------------------------------------------------------|------|
| Modification du profil  | Mise à jour email, username, mot de passe encodé BCrypt      | Fait |
| Déconnexion             | Suppression du token JWT côté client                         | Fait |
| Consultation des thèmes | Liste de tous les sujets avec statut d'abonnement            | Fait |
| Abonnement à un thème   | Depuis la page Thèmes — bouton devient 'Déjà abonné'         | Fait |
| Désabonnement           | Depuis la page Profil                                        | Fait |
| Fil d'actualité (feed)  | Articles triés par date décroissante des thèmes abonnés      | Fait |
| Tri du fil d'actualité  | Tri croissant / décroissant par date                         | Fait |
| Création d'article      | Choix du thème, titre, contenu — auteur et date automatiques | Fait |

|                           |                                                                |      |
|---------------------------|----------------------------------------------------------------|------|
| Consultation d'un article | Thème, titre, auteur, date, contenu et commentaires            | Fait |
| Ajout d'un commentaire    | Contenu uniquement — auteur et date automatiques, non récursif | Fait |
| Responsive design         | Application fonctionnelle sur mobile et desktop                | Fait |

## 2. Architecture et conception technique

### 2.1 Schéma global de l'architecture

#### Architecture du Front

Ci-dessous, nous avons la structure des dossiers du projet Angular :

| Dossier / Fichier          | Rôle                                                                                     |
|----------------------------|------------------------------------------------------------------------------------------|
| src/app/pages/             | Composants de pages (feed, login, register, profile, subjects, create-post, post-detail) |
| src/app/core/services/     | Services HTTP (AuthService, UserService, PostService, SubjectService)                    |
| src/app/core/interceptors/ | jwtInterceptor — injection automatique du token Bearer                                   |
| src/app/guards/            | authGuard (routes protégées), unauthGuard (redirection si connecté)                      |
| src/app/shared/models/     | Interfaces TypeScript (AuthResponse, PostResponse, SubjectResponse, UserResponse...)     |
| src/app/shared/components/ | Composants réutilisables (Navbar)                                                        |
| app.routes.ts              | Configuration centralisée du routage.                                                    |

## Architecture back-end (Spring Boot)

Nous avons la structure de l'API Spring Boot. :

| Package                     | Rôle                                                                |
|-----------------------------|---------------------------------------------------------------------|
| controller/                 | REST controllers exposant les endpoints (Auth, Post, Subject, User) |
| service/ + service/impl/    | Interfaces + implémentations de la logique métier                   |
| repository/                 | Interfaces Spring Data JPA (CRUD + requêtes personnalisées)         |
| dto/request + dto/response/ | Objets de transfert (Request/Response DTOs) — séparation modèle/API |
| mapper/                     | MapStruct — conversion automatique entité ↔ DTO                     |
| model/                      | Entités JPA (User, Post, Subject, Comment)                          |
| mapper/                     | MapStruct — conversion automatique entité ↔ DTO                     |
| security/                   | JwtFilter, JwtTokenProvider, UserDetailsService, SecurityConfig     |
| config/                     | Configuration Spring (CORS, Security, Beans)                        |

## 2.2 Choix techniques

Le tableau ci-dessous contient les justification pour chaque choix structurel :

| Élément choisi                      | Type                | Lien documentation                                                                          | Objectif                                 | Justification                                                                                                                    |
|-------------------------------------|---------------------|---------------------------------------------------------------------------------------------|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Angular 21                          | Framework front-end | <a href="https://angular.dev">angular.dev</a>                                               | SPA réactive avec composants standalone  | Imposé par le cahier des charges ORION. Standalone components, signals, inject() — architecture moderne et testable.             |
| Spring Boot 4.0.2                   | Framework back-end  | <a href="https://spring.io/projects/spring-boot">spring.io/projects/spring-boot</a>         | API REST robuste avec auto-configuration | Choix initial d'Heidi, cohérent avec les contraintes Java + Spring. Écosystème mature, Spring Data JPA intégré.                  |
| MySQL 8                             | Base de données     | <a href="https://dev.mysql.com/doc">dev.mysql.com/doc</a>                                   | Persistence relationnelle des données    | Choix d'Heidi. Base relationnelle adaptée aux relations User-Subject-Post-Comment. Spring Data JPA + Hibernate gèrent le schéma. |
| Spring Security + JWT (JJWT 0.11.5) | Sécurité            | <a href="https://spring.io/projects/spring-security">spring.io/projects/spring-security</a> | Authentification stateless sécurisée     | JWT préféré à Basic Auth pour la persistance de session côté client sans état serveur. Adapté au MVP interne.                    |

|                 |                   |                   |                                                      |                                                                                                                                           |
|-----------------|-------------------|-------------------|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Lombok 1.18.32  | Bibliothèque Java | projectlombok.org | Réduction du boilerplate (getters, constructeurs...) | Standard de l'écosystème Spring. Réduit la verbosité Java sans impact sur les performances.                                               |
| MapStruct 1.6.3 | Mapper Java       | mapstruct.org     | Conversion entité ↔ DTO sans code manuel             | Génération à la compilation (pas de réflexion). Plus performant et typé que ModelMapper. Intégration Lombok via lombok-mapstruct-binding. |
| H2 (tests)      | BDD en mémoire    | h2database.com    | Tests d'intégration sans MySQL                       | Légère, embarquée, aucune installation requise en CI. Remplace MySQL en                                                                   |

|                                |                 |                  |                                       |                                                                                               |
|--------------------------------|-----------------|------------------|---------------------------------------|-----------------------------------------------------------------------------------------------|
|                                |                 |                  |                                       | scope test.                                                                                   |
| JUnit 5 + Spring Security Test | Tests back-end  | junit.org/junit5 | Tests unitaires et d'intégration Java | Standard de facto Spring Boot. Spring Security Test permet de tester les endpoints sécurisés. |
| Jest + jest-preset-angular     | Tests front-end | jestjs.io        | Tests unitaires Angular avec coverage | Plus rapide que Karma/Jasmine. Compatible Angular 21 via jest-preset-angular.                 |



|                         |                       |                                                                                         |                                          |                                                                                                    |
|-------------------------|-----------------------|-----------------------------------------------------------------------------------------|------------------------------------------|----------------------------------------------------------------------------------------------------|
| JaCoCo 0.8.11           | Couverture back-end   | <a href="http://jacoco.org">jacoco.org</a>                                              | Mesure et seuil de couverture à 70%      | Plugin Maven natif Spring Boot. Seuil 70% configuré sur les packages service et controller.        |
| Architecture en couches | Pattern architectural |                                                                                         | Séparation Controller/Service/Repository | Respect des principes SOLID. Facilite les tests unitaires par mocking de couches. Standard Spring. |
| BCrypt                  | Chiffrement           | <a href="https://www.npmjs.com/package/bcrypt">https://www.npmjs.com/package/bcrypt</a> | Hachage des mots de passe                | Standard sécuritaire. Résistant aux attaques par force brute grâce au salt automatique.            |

## 2.3 API et schémas de données

Ci dessous nous avons chaque endpoint de l'API :

| Endpoint           | Méthode | Description                   | Corps / Réponse                                                  |
|--------------------|---------|-------------------------------|------------------------------------------------------------------|
| /api/auth/register | POST    | Inscription d'un utilisateur  | Body: {username, email, password} → {token, id, username, email} |
| /api/auth/login    | POST    | Connexion (email ou username) | Body: {usernameOrEmail, password} → {token, id, username, email} |

|                                |      |                                    |                                                                                    |
|--------------------------------|------|------------------------------------|------------------------------------------------------------------------------------|
| /api/users/me                  | GET  | Profil de l'utilisateur connecté   | Header: Bearer token → {id, username, email, subscriptions[]}                      |
| /api/users/me                  | PUT  | Mise à jour du profil              | Body: {username?, email?, password?} → UserResponse                                |
| /api/subjects                  | GET  | Liste de tous les thèmes           | Header: Bearer → [{id, name, description, subscribed}]                             |
| /api/subjects/{id}/subscribe   | POST | Abonnement à un thème              | Header: Bearer → SubjectResponse {subscribed: true}                                |
| /api/subjects/{id}/unsubscribe | POST | Désabonnement d'un thème           | Header: Bearer → SubjectResponse {subscribed: false}                               |
| /api/subjects/me               | GET  | Mes abonnements                    | Header: Bearer → [SubjectResponse]                                                 |
| /api/posts/feed                | GET  | Fil d'actualité personnel          | Header: Bearer → [{id, title, content, authorUsername, subjectName, creationDate}] |
| /api/posts                     | POST | Créer un article                   | Body: {title, content, subjectId} → PostResponse                                   |
| /api/posts/{id}                | GET  | Détail d'un article + commentaires | Header: Bearer → PostResponse {..., comments[]}                                    |

|                          |      |                           |                                                                             |
|--------------------------|------|---------------------------|-----------------------------------------------------------------------------|
| /api/posts/{id}/comments | POST | Ajouter un commentaire    | Body: {content} →<br>CommentResponse {id,<br>content, author, creationDate} |
| /api/posts/{id}/comments | GET  | Commentaires d'un article | Header: Bearer →<br>[CommentResponse]                                       |

#### Règles de validation :

- Mot de passe : plus de 8 caractères, au moins 1 majuscule, 1 minuscule, 1 chiffre, 1 caractère spécial.
- Email : format valide, unique en base.
- Username : unique en base, plus de 3 caractères .
- Commentaire non récursif .
- Auteur et date des articles/commentaires définis automatiquement côté serveur.

## 3. Tests, performance et qualité

### 3.1 Stratégie de test

La stratégie de test couvre les deux parties de l'application avec des outils adaptés à chaque environnement :

| Type de test              | Outil / Framework | Portée                                                                                    | Résultats                                         |
|---------------------------|-------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------|
| Test unitaire<br>back-end | JUnit 5 + Mockito | Services (AuthServiceImpl,<br>PostServiceImpl,<br>SubjectServiceImpl,<br>UserServiceImpl) | Couverture $\geq$ 70% (seuil<br>JaCoCo configuré) |

|                            |                               |                                                                                                  |                                                  |
|----------------------------|-------------------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------|
| Test e2e                   | Manuel (Postman)              | Flux complets : inscription<br>→ connexion → création<br>article → commentaire                   | Scénarios clés validés via<br>collection Postman |
| Test unitaire<br>front-end | Jest +<br>jest-preset-angular | Services Angular (auth,<br>user, post, subject),<br>guards, intercepteur,<br>composants de pages | Couverture mesurée via jest<br>--coverage        |

### Instructions pour exécuter les tests:

#### Back-end (Maven) :

```
# Lancer tous les tests unitaires et
mvn test
# Lancer les tests + vérification du seuil JaCoCo 70%
mvn verify
# Rapport de couverture (HTML généré dans target/site/jacoco/index.html)
mvn jacoco:report
```

#### Front-end (Angular / Jest) :

```
# Lancer tous les tests
npm test
# Rapport de couverture (HTML dans coverage/)
npm run coverage
# Tests par module (exemples)
npm run test:auth # Service d'authentification
npm run test:post # Service des articles
npm run test:subject # Service des thèmes
```

### 3.2 Rapport de performance et optimisation

Les actions décrites ci-dessous ont été menées pour améliorer la qualité et les performances de l'application :

| Axe d'optimisation               | Problème identifié                                    | Action corrective                                                                                                                               | Résultat                                                                                     |
|----------------------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Encodage des mots de passe       | Risque de stockage en clair si la validation échoue   | Double vérification BCrypt avant et après sauvegarde en BDD (startsWith '\$2')                                                                  | Sécurité renforcée — exception explicite si hash invalide                                    |
| Détection de changements Angular | Composants non mis à jour après appel HTTP asynchrone | Injection et appel manuel de <code>ChangeDetectorRef.detectChanges()</code>                                                                     | Affichage temps réel des données sans zone.js issues                                         |
| Exclusion JaCoCo                 | Les DTOs, mappers, modèles faussent la couverture     | Exclusion des packages <code>dto/</code> , <code>mapper/</code> , <code>model/</code> , <code>security/</code> , <code>config/</code> de JaCoCo | Couverture représentative des couches testables uniquement                                   |
| Intercepteur JWT Angular         | Token manquant sur certaines requêtes HTTP            | <code>jwtInterceptor</code> injecté globalement via <code>provideHttpClient(withInterceptors([jwtInterceptor]))</code>                          | Toutes les requêtes protégées reçoivent automatiquement le header <code>Authorization</code> |

### 3.3 Revue technique

#### Points forts

- Architecture en couches stricte : séparation Controller / Service (interface + impl) / Repository. Facilite les tests unitaires par mocking.
- Utilisation de DTOs dédiés (Request/Response) : aucune entité JPA exposée directement dans l'API. Sécurité et découplage garantis.
- MapStruct pour les conversions : typé, rapide, généré à la compilation — pas de magie par réflexion.
- Sécurité JWT robuste : filtre dédié (JwtFilter extends OncePerRequestFilter), token validé systématiquement avant chaque requête non publique.
- Intercepteur Angular centralisé : injection automatique du token sur 100% des requêtes HTTP sortantes sans duplication de code.
- Standalone components Angular : architecture moderne, plus légère, sans NgModule.

#### Points à améliorer / dette technique

- Gestion des erreurs côté back-end : les RuntimeException génériques devraient être remplacées par des exceptions métier typées.
- Pas de pagination sur les listes : le feed, les articles et les commentaires sont retournés en totalité. À prévoir pour la scalabilité.

#### Actions correctives appliquées

- Encodage BCrypt : double vérification avant et après sauvegarde pour garantir que le mot de passe n'est jamais stocké en clair.
- Exclusion JaCoCo : les packages non testables (DTOs, modèles, mappers) sont exclus du rapport de couverture pour obtenir un taux représentatif.

## 4. Documentation utilisateur et supervision

### 4.1 FAQ utilisateur

**Q : Comment créer un compte ?**

R : Depuis la page d'accueil, cliquez sur « S'inscrire ». Renseignez un nom d'utilisateur, une adresse email valide et un mot de passe (8 caractères minimum, avec au moins une majuscule, une

minuscule, un chiffre et un caractère spécial). Validez le formulaire pour accéder directement à l'application.

**Q : Comment me connecter ?**

R : Depuis la page de connexion, saisissez votre email ou votre nom d'utilisateur, ainsi que votre mot de passe. Votre session sera maintenue automatiquement (token JWT stocké localement).

**Q : Comment m'abonner à un thème ?**

R : Accédez à la page « Thèmes » via le menu de navigation. Cliquez sur « S'abonner » pour le thème souhaité. Le bouton devient inactif et affiche « Déjà abonné » pour confirmer votre abonnement.

**Q : Comment me désabonner d'un thème ?**

R : Rendez-vous sur votre page de profil. Dans la section « Mes abonnements », cliquez sur « Se désabonner » en regard du thème concerné.

**Q : Mon fil d'actualité est vide, pourquoi ?**

R : Le fil d'actualité affiche uniquement les articles des thèmes auxquels vous êtes abonné. Abonnez-vous à au moins un thème depuis la page « Thèmes » pour voir apparaître des articles.

**Q : Comment créer un article ?**

R : Cliquez sur l'icône de création (stylo) dans la barre de navigation. Choisissez un thème dans la liste déroulante, saisissez un titre (3 caractères minimum) et le contenu de votre article (10 caractères minimum). Votre nom et la date sont ajoutés automatiquement.

**Q : Comment commenter un article ?**

R : Ouvrez un article en cliquant dessus depuis le fil d'actualité. Faites défiler jusqu'à la section commentaires, saisissez votre commentaire et validez. Votre nom et la date sont ajoutés automatiquement.

**Q : Comment modifier mon profil ?**

R : Accédez à la page « Profil » via le menu. Modifiez votre nom d'utilisateur, email et/ou mot de passe, puis cliquez sur « Enregistrer ». Laissez le champ mot de passe vide si vous ne souhaitez pas le modifier.

## 4.2 Supervision et tâches déléguées à l'IA

| Tâche déléguée                       | Outil              | Objectif                                                                                        | Vérification effectuée                                                                                                                  |
|--------------------------------------|--------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Génération des DTOs Request/Response | Claude (Anthropic) | Gain de temps sur les objets de transfert répétitifs                                            | Vérification des champs par rapport aux spécifications fonctionnelles et aux entités JPA                                                |
| Squelettes de tests unitaires JUnit  | Claude (Anthropic) | Structure de base des classes de test avec @BeforeEach, mocks Mockito                           | Validation de chaque assertion, correction des cas manquants (cas d'erreur)                                                             |
| Tests Jest (Angular)                 | Claude (Anthropic) | Generation des test jest                                                                        | Vérification du mock des dépendances, correction des imports, exécution avec Jest pour vérifier la couverture et correction des erreurs |
| Debugging (Java / Angular)           | Claude Anthopic    | Obtenir des pistes de résolution pour des erreurs spécifiques, logs ou comportements inattendus | Validation croisée avec la documentation officielle, reproduction de la solution en local, tests manuels pour confirmer la correction   |

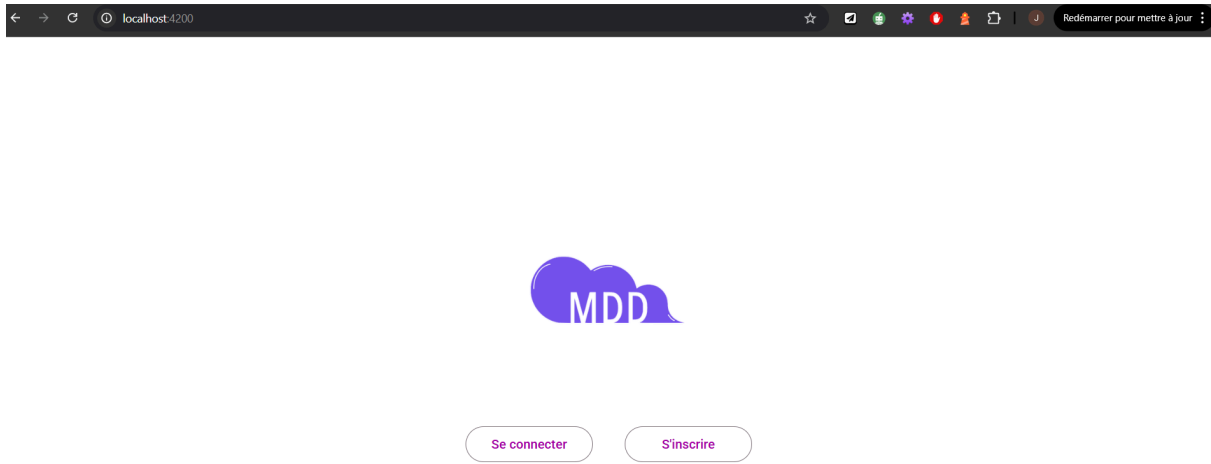
**Posture de supervision adoptée :** chaque livrable IA a été relu, testé et validé avant intégration. Aucun code généré n'a été utilisé sans compréhension préalable de son fonctionnement. Les erreurs détectées ont été corrigées manuellement.



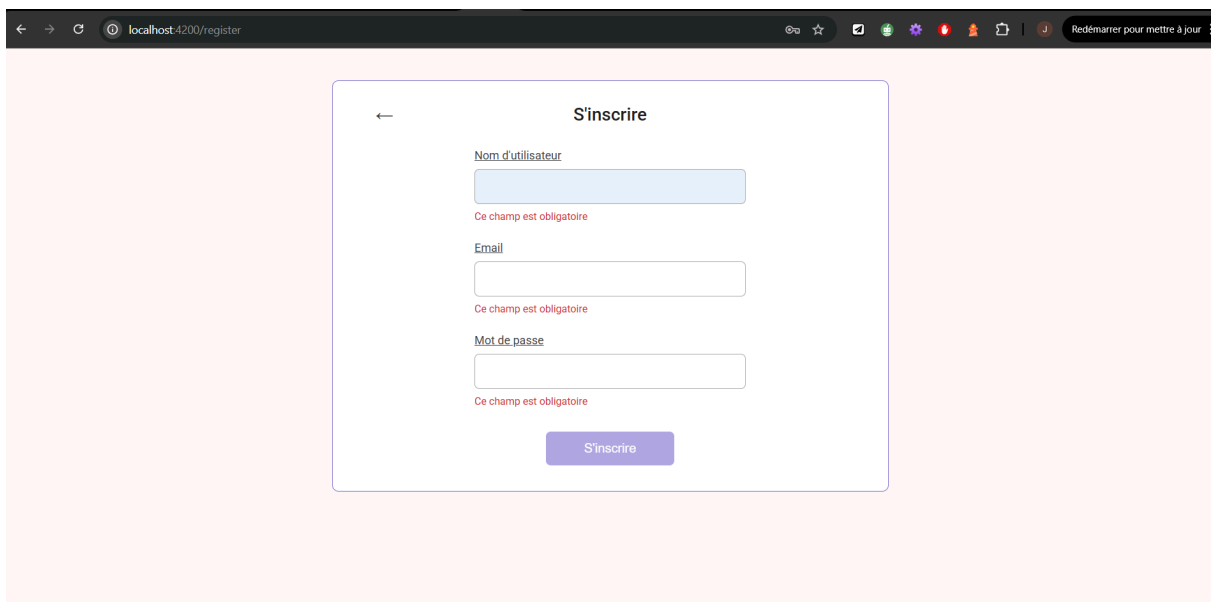
## 5. Annexes

### Annexe 1 — Captures d'écran de l'UI

Page d'accueil (non connecté) :



Page d'inscription :



Page de connexion :



← **Se connecter**

E-mail ou nom d'utilisateur

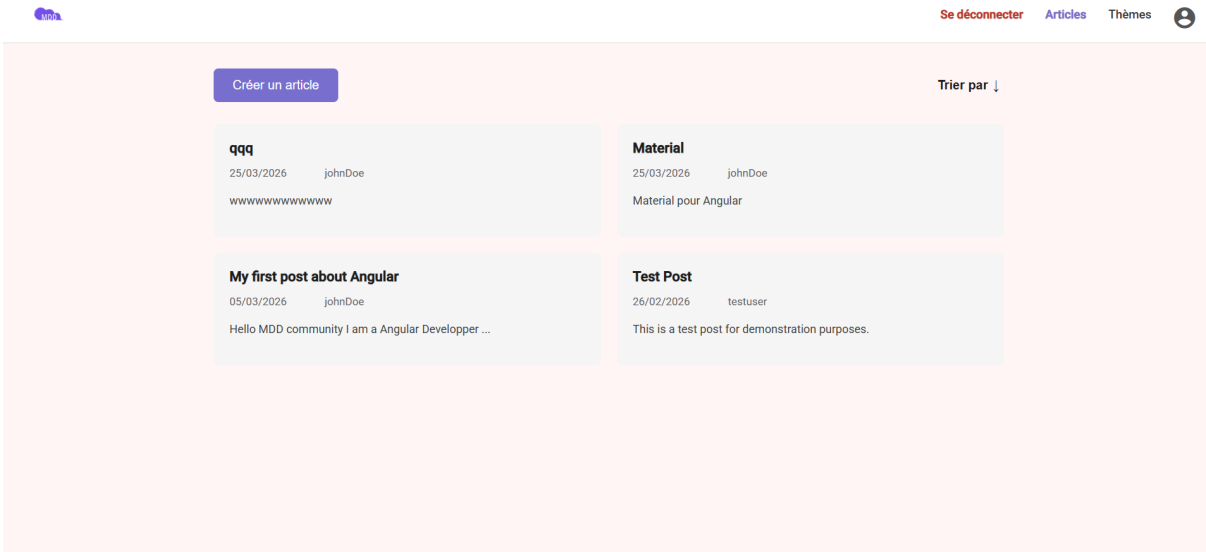
Ce champ est obligatoire

Mot de passe

Ce champ est obligatoire

Se connecter

Fil d'actualité (feed) :



Créer un article

Se déconnecter Articles Thèmes

Trier par ↓

**qqq**

25/03/2026 johnDoe

www

**Material**

25/03/2026 johnDoe

Material pour Angular

**My first post about Angular**

05/03/2026 johnDoe

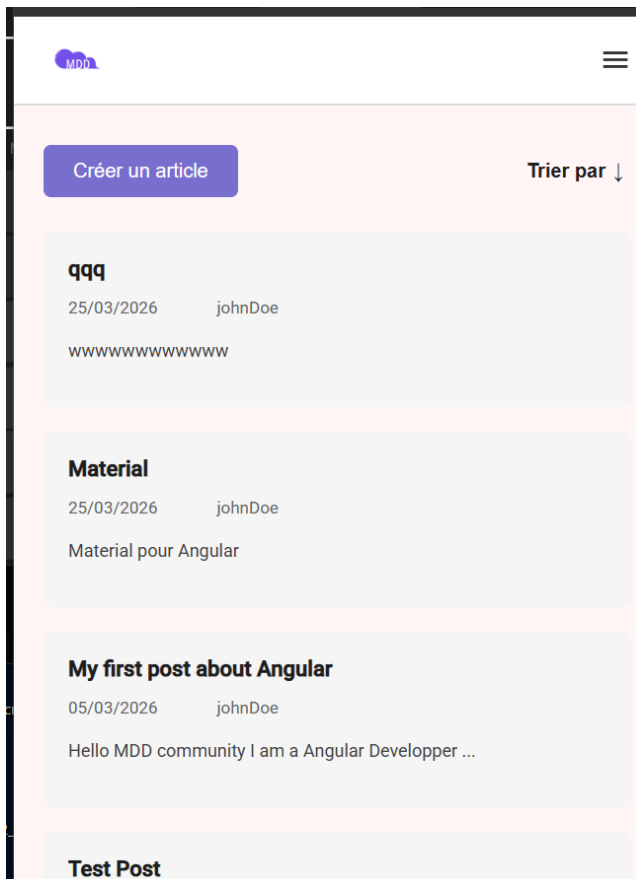
Hello MDD community I am a Angular Developer ...

**Test Post**

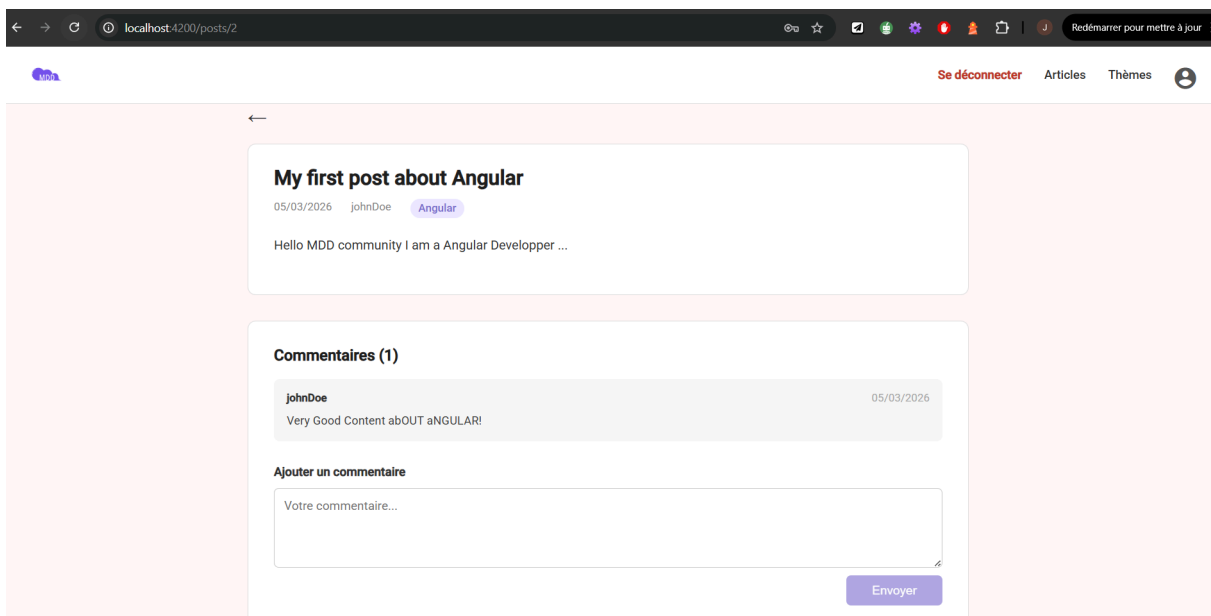
26/02/2026 testuser

This is a test post for demonstration purposes.

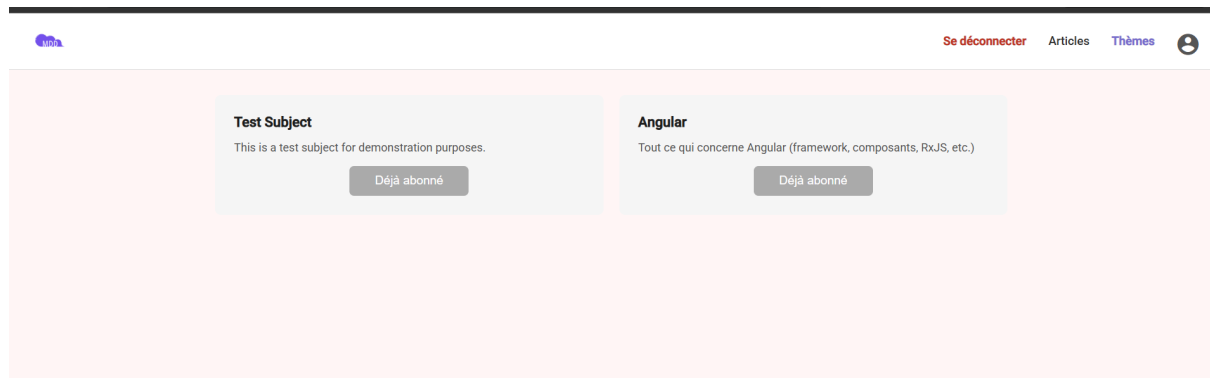
## Fil d'actualité (Mobile) :



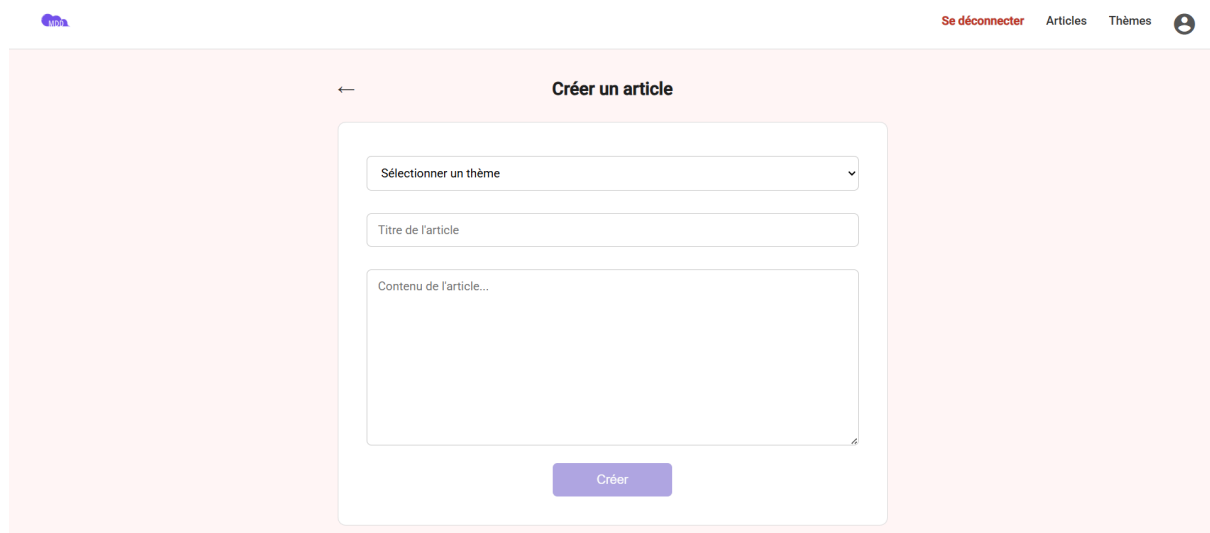
## Détail d'un article :



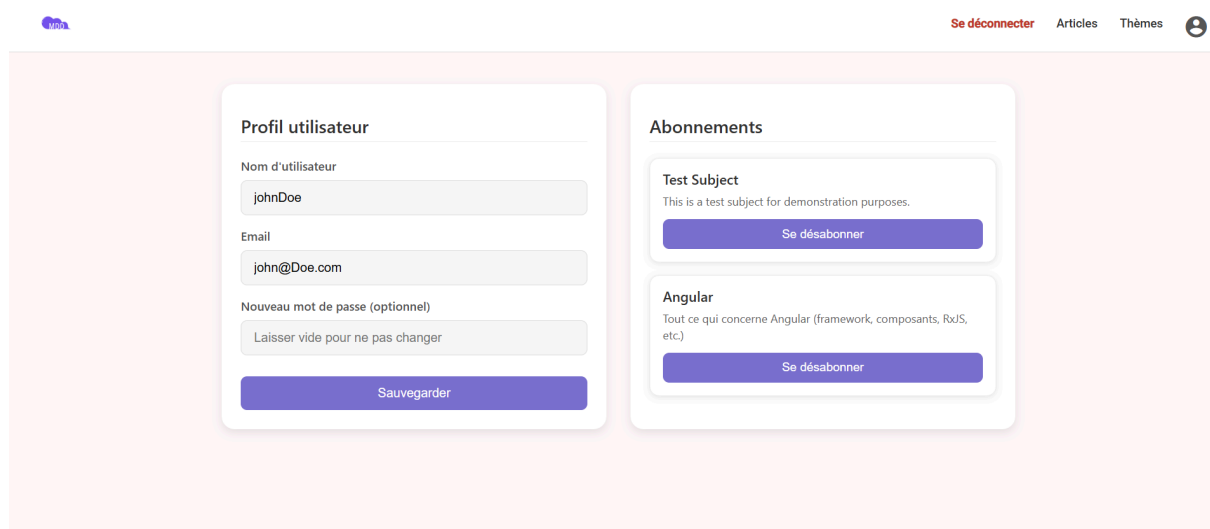
## Page des thèmes :



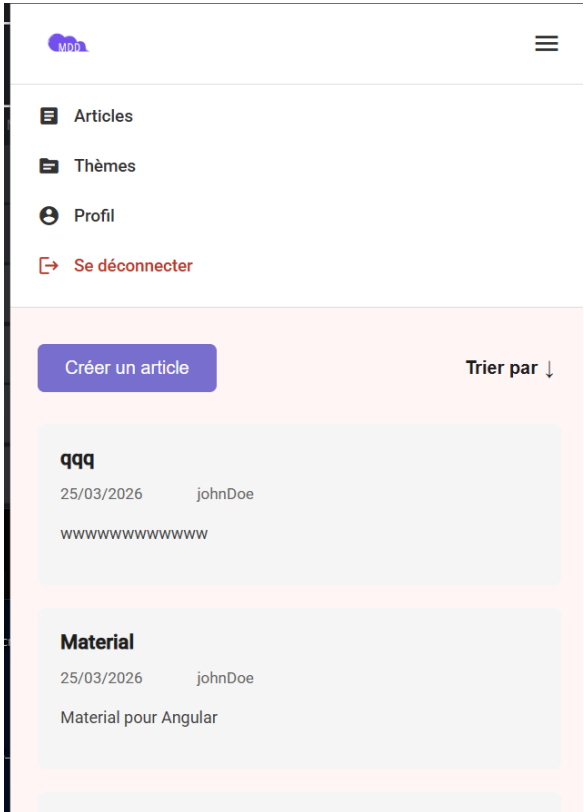
## Création d'article :



## Page de profil



Menu (Mobile ) :



Annexe 2 — Analyse des besoins front-end

Correspondance entre les spécifications fonctionnelles et les composants Angular implémentés :

| Besoin fonctionnel   | Composant Angular | Service utilisé        | Guard |
|----------------------|-------------------|------------------------|-------|
| Accueil non connecté | Landing           | —                      | —     |
| Inscription          | Register          | AuthService.register() | —     |

|                             |                |                                                                         |           |
|-----------------------------|----------------|-------------------------------------------------------------------------|-----------|
| Connexion                   | Login          | AuthService.login()                                                     | —         |
| Fil d'actualité             | Feed           | PostService.getFeed()                                                   | authGuard |
| Liste des thèmes            | Subjects       | SubjectService.getAll(),<br>subscribe(), unsubscribe()                  | authGuard |
| Détail d'un article         | PostDetail     | PostService.getByld(),<br>CommentService                                | authGuard |
| Création d'article          | CreatePost     | PostService.createPost(),<br>getSubjects()                              | authGuard |
| Profil utilisateur          | Profile        | UserService.getMe(),<br>updateMe(),<br>SubjectService.unsubscri<br>be() | authGuard |
| Injection JWT automatique   | jwtInterceptor | localStorage.getItem('tok<br>en')                                       | —         |
| Redirection si non connecté | authGuard      | localStorage.getItem('tok<br>en')                                       | —         |

## Annexe 3 — Définition des données

### Entités et règles de validation

| Entité  | Champ        | Type          | Contrainte / Règle                                                   |
|---------|--------------|---------------|----------------------------------------------------------------------|
| User    | id           | Long          | Clé primaire, auto-incrémentée                                       |
| User    | username     | String        | Non nul, unique, 3+ caractères                                       |
| User    | email        | String        | Non nul, unique, format email valide                                 |
| User    | password     | String        | Non nul, hashé BCrypt (\$2a\$...), 8+ chars, maj+min+chiffre+spécial |
| Subject | id           | Long          | Clé primaire, auto-incrémentée                                       |
| Subject | name         | String        | Non nul, unique                                                      |
| Subject | description  | String        | Optionnel                                                            |
| Post    | id           | Long          | Clé primaire, auto-incrémentée                                       |
| Post    | title        | String        | Non nul, 3+ caractères                                               |
| Post    | content      | String        | Non nul, 10+ caractères                                              |
| Post    | creationDate | LocalDateTime | Définie automatiquement côté serveur                                 |

|         |              |               |                                      |
|---------|--------------|---------------|--------------------------------------|
| Post    | author       | User          | FK → users.id, non nul               |
| Post    | subject      | Subject       | FK → subjects.id, non nul            |
| Comment | id           | Long          | Clé primaire, auto-incrémentée       |
| Comment | content      | String        | Non nul                              |
| Comment | creationDate | LocalDateTime | Définie automatiquement côté serveur |
| Comment | post         | Post          | FK → posts.id, non nul, non récursif |
| Comment | author       | User          | FK → users.id, non nul               |

## Annexe 4 — Rapport de couverture et de tests

JaCoCo (back-end) — seuil minimum de 70% :

| mdd                        |                                                                                     |      |                                                                                     |      |             |              |                |                |  |  |  |
|----------------------------|-------------------------------------------------------------------------------------|------|-------------------------------------------------------------------------------------|------|-------------|--------------|----------------|----------------|--|--|--|
| mdd                        |                                                                                     |      |                                                                                     |      |             |              |                |                |  |  |  |
| Element                    | Missed Instructions                                                                 | Cov. | Missed Branches                                                                     | Cov. | Missed Cxty | Missed Lines | Missed Methods | Missed Classes |  |  |  |
| com.orion.mdd.controller   |  | 72 % |  | 44 % | 13 34       | 16 55        | 6 25           | 1 4            |  |  |  |
| com.orion.mdd.service.impl |  | 96 % |  | 71 % | 13 64       | 5 155        | 0 41           | 0 5            |  |  |  |
| com.orion.mdd              |  | 37 % |  | n/a  | 1 2         | 2 3          | 1 2            | 0 1            |  |  |  |
| Total                      | 96 of 1 014                                                                         | 90 % | 23 of 64                                                                            | 64 % | 27 100      | 23 213       | 7 68           | 1 10           |  |  |  |



Jest (front-end) — couverture via --coverage :

| File                             | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s |
|----------------------------------|---------|----------|---------|---------|-------------------|
| All files                        | 97.98   | 93.18    | 94.04   | 97.8    |                   |
| app                              | 100     | 100      | 100     | 100     |                   |
| app.ts                           | 100     | 100      | 100     | 100     |                   |
| app/core/interceptors            | 100     | 100      | 100     | 100     |                   |
| jwt-interceptor.ts               | 100     | 100      | 100     | 100     |                   |
| app/core/services                | 90.9    | 100      | 75      | 89.47   |                   |
| auth.ts                          | 96.87   | 100      | 100     | 96.55   | 88                |
| post.ts                          | 66.66   | 100      | 28.57   | 61.53   | 18-30             |
| subject.ts                       | 100     | 100      | 100     | 100     |                   |
| user.ts                          | 100     | 100      | 100     | 100     |                   |
| app/guards                       | 100     | 100      | 100     | 100     |                   |
| auth-guard.ts                    | 100     | 100      | 100     | 100     |                   |
| unauth-guard.ts                  | 100     | 100      | 100     | 100     |                   |
| app/pages/create-post            | 100     | 100      | 100     | 100     |                   |
| create-post.ts                   | 100     | 100      | 100     | 100     |                   |
| app/pages/feed                   | 100     | 100      | 100     | 100     |                   |
| feed.ts                          | 100     | 100      | 100     | 100     |                   |
| app/pages/landing                | 100     | 100      | 100     | 100     |                   |
| landing.ts                       | 100     | 100      | 100     | 100     |                   |
| app/pages/login                  | 100     | 100      | 100     | 100     |                   |
| login.ts                         | 100     | 100      | 100     | 100     |                   |
| app/pages/post-detail            | 100     | 80       | 100     | 100     |                   |
| post-detail.ts                   | 100     | 80       | 100     | 100     | 59                |
| app/pages/profile                | 100     | 75       | 100     | 100     |                   |
| profile.ts                       | 100     | 75       | 100     | 100     | 109               |
| app/pages/register               | 95.83   | 93.33    | 100     | 95.45   |                   |
| register.ts                      | 95.83   | 93.33    | 100     | 95.45   | 75-76             |
| app/pages/subjects               | 100     | 100      | 100     | 100     |                   |
| subjects.ts                      | 100     | 100      | 100     | 100     |                   |
| app/shared/components/navbar     | 100     | 100      | 100     | 100     |                   |
| navbar.ts                        | 100     | 100      | 100     | 100     |                   |
| Test Suites: 17 passed, 17 total |         |          |         |         |                   |
| Tests: 210 passed, 210 total     |         |          |         |         |                   |